

Chaitin–Gödel I in BRA4: a genuine $\mathbf{Fun}_1$ diagonal with an internal implication

1 Chaitin’s argument, recalled

Chaitin’s proof of Gödel’s first incompleteness theorem [1, 2] runs as follows. Fix a sound theory T that interprets enough arithmetic, and fix a universal partial computable interpreter U with a Kolmogorov-complexity bound $K_U(x) := \min\{|p| : U(p) \downarrow = x\}$. There is a constant c (depending only on T and U) such that, for every L^* and every integer x' ,

$$T \vdash K_U(\underline{x'}) > L^* \implies T \text{ is inconsistent, provided } L^* > c + \log L^*.$$

The contradiction is exhibited by a single short program g_{L^*} : enumerate $\mathbf{thmT}(0), \mathbf{thmT}(1), \dots$ until a proof of the form $\mathbf{code}K_U(?) > L^*$ is found, then output the witnessed integer x' . The size of g_{L^*} is $c + \log L^*$, strictly below L^* , so the very witness x' that T proves to be K_U -incompressible is itself g_{L^*} -output — a contradiction to the size lemma internal to T .

2 The BRA4 statement

The formalisation packages Chaitin’s argument as a single closed T -derivation: it produces, from any putative T -proof that $\mathbf{thmT} w$ asserts incompressibility of its own read-back, a T -proof that a constructed code is a proof of $0 = 1$. Two design choices make this statement sharp.

1. The “construct the short program g_{L^*} ” step is a genuine BRA combinator $\mathbf{G} : \mathbf{Fun}_1$, an honest object-level program, not a meta-level term transformer. The constructed proof code is literally $\mathbf{ap}_1 \mathbf{G} w$.
2. The “if ... then inconsistent” step is an *internal* Hilbert implication $\Rightarrow$ inside a single $\mathbf{Deriv}$, not a meta-level function $\mathbf{Deriv} X \rightarrow \mathbf{Deriv} Y$. The deduction content lives inside the object logic.

The headline theorem (`BRA4/CgFunFun1.agda`, `chaitinG1_Fun1_all`) is:

`chaitinG1_Fun1_all` : $(w : \mathbf{Term}) \rightarrow$

$$\mathbf{Deriv} \left(\mathbf{eqF} (\mathbf{ap}_1 \mathbf{thmT} w) (\mathbf{ap}_1 (\mathbf{Kcode}_{L^*}) (\mathbf{ap}_1 \mathbf{outKdef}_{L^*} w)) \Rightarrow \mathbf{eqF} (\mathbf{ap}_1 \mathbf{thmT} (\mathbf{ap}_1 \mathbf{G} w)) \mathbf{codeFalse} \right)$$

Here w is an *arbitrary* $\mathbf{Term}$: no closure hypothesis, no $\mathbf{isNat}$ integrality predicate, no $\mathbf{Con}(T)$ premise, and no closedness bookkeeping. In words: *for every* w , T proves “if $\mathbf{thmT} w$ is the incompressibility statement $\mathbf{Kcode}_{L^*}$ about its own read-back $\mathbf{outKdef}_{L^*} w$, then the code $\mathbf{ap}_1 \mathbf{G} w$ is a T -proof of $0 = 1$ ”.

The antecedent $\mathbf{eqF} (\mathbf{ap}_1 \mathbf{thmT} w) (\mathbf{Kcode}_{L^*} (\mathbf{outKdef}_{L^*} w))$ is the firing condition in $\mathbf{num}$ -raw form: the subject slot is $\mathbf{num} x'$ for an arbitrary $\mathbf{Term} x'$, with no integrality required to align with the read-back. Whether $\mathbf{thmT} w$ has the form $\mathbf{Kcode}_{L^*} (-)$ is read off internally by the BRA function $\mathbf{outKdef}_{L^*} = \mathbf{decode} \circ \mathbf{projKdef} \circ \mathbf{thmT}$, so the witnessed integer is forced to be $\mathbf{outKdef}_{L^*} w$.

3 The diagonal program G

3.1 Bracket abstraction: combinatory completeness

The short program is constructed by a one-variable bracket-abstraction DSL (`BRA4/AbsFun1.agda`). An expression $e : \text{Exp}$ is a Term-valued expression in one distinguished variable, built from

- the variable leaf `evar`;
- constant leaves `econst c` with $c : \text{Term}$ var-free;
- object application nodes `eap1 f (f : Fun1)` and `eap2 g (g : Fun2)`.

Two maps interpret `Exp`:

`denote : Exp → Term → Term` (plug x for `evar`), `compile : Exp → Fun1` (bracket-abtract `evar`),

with `compile evar = u`, `compile (eap1 f e) = compose1U f (compile e)`, and `compile (eap2 g a b) = C g (compile a) (compile b)`. The combinatory-completeness fact is a *proved* `Deriv`:

$$\text{compile_eq} : (e : \text{Exp}) (x : \text{Term}) \rightarrow \text{Deriv} (\text{eqF} (\text{ap}_1 (\text{compile } e) x) (\text{denote } e x)),$$

by induction on e from the axioms `ax_u` (variable), `constTermFun1_eq` (constant), and `axComp/axComp2` with `cong` congruences (application nodes).

Remark 1. *The factorisation is genuinely a `Deriv`, not a meta-level (Agda) equality. `ap1 (compile e) x` has head constructor `ap1`, while `denote e x` generally has a different head (e.g. `ap2 Pair`): they are distinct Term normal forms. The combinator semantics lives only in `Deriv`, through the `ax_C/axComp/ax_u` axioms. There is no definitional collapse to exploit.*

3.2 The body expression

The Term that Chaitin’s construction produces from w is a fixed object-level combinator context $E[\cdot]$ with a single w -dependent slot. Every node of E is an `ap1/ap2` of an object `Fun1/Fun2` (`gRec`, `sigma`, `pi`, `num`, `Pair`, the fuel combinators `fuelMu`, `fuelG`, the read-back `outKdefL*`, `runProg`, `s`) or a var-free constant; the proof-assembly builders (`cmp`, `clmp`, `cNeg`, `cEqTm`, `cAp1f`, `cAp2f`, `exfProof` and its `axK/axNeg/axS/liftP/bComb` chains) are themselves pure `ap2 Pair`-skeletons of their arguments. Hence $E[\cdot]$ is a one-variable `Exp`, transcribed node-for-node into `bodyExp` (`BRA4/CgFunFun1.agda`) using `Exp` smart constructors that mirror each builder (`ecmpE`, `eexfProofE`, ...). The diagonal program and its defining fact are

$$G := \text{compile bodyExp} : \text{Fun}_1, \quad \text{gFact } w := \text{compile_eq bodyExp} (\text{closeW } w).$$

Here `closeW w := substT 0 O (substT 1 O w)` forces the variables 0 and 1 in w to O ; on the denotation side `denote bodyExp (closeW w)` is the Chaitin Term verbatim, because the private constants in `bodyExp` are replicated exactly. Thus

$$\text{gFact} : (w : \text{Term}) \rightarrow \text{Deriv} (\text{eqF} (\text{ap}_1 G (\text{closeW } w)) (\text{denote bodyExp} (\text{closeW } w))),$$

a proved object equality identifying the program image `ap1 G` with the Chaitin construction.

4 The internal implication $\Rightarrow$

The “if firing condition then inconsistent” step is an object-level implication, obtained by internalising the deduction theorem (BRA4/CgFalseImp.agda). The parametric form carries an arbitrary antecedent context $R : \text{Formula}$:

$$\begin{aligned} \text{cgFalseImp} : (R : \text{Formula}) (w : \text{Term}) \rightarrow \\ & (\text{closedness of } R \text{ at vars } 0, 1) \rightarrow \\ & \text{Deriv } (R \Rightarrow \text{eqF } (\text{ap}_1 \text{ thmT } (\text{closeW } w)) (\text{Kcode}_{L^*} (\text{outKdef}_{L^*} (\text{closeW } w)))) \rightarrow \\ & \text{Deriv } (R \Rightarrow \text{eqF } (\text{ap}_1 \text{ thmT } (\text{cgFun } w)) \text{codeFalse}). \end{aligned}$$

The implication is a single Deriv over $\Rightarrow$; no meta-level arrow $\text{Deriv } X \rightarrow \text{Deriv } Y$ ever appears. The antecedent context R threads through the ruleIndNat motive $Q := R \Rightarrow P$ used in the operational core, and through the substitution bridges, with the closedness hypotheses on R supplying what those steps need.

Setting R to the firing condition itself and applying internal reflexivity gives the un-parametrised form cgFalseImpDed , and instantiating at the fresh witness $w := \text{var } 2$ gives the closed Hilbert formula cgFalseImpAtVar2 :

$$\text{Deriv } (\text{eqF } (\text{ap}_1 \text{ thmT } v_2) (\text{Kcode}_{L^*} (\text{outKdef}_{L^*} v_2)) \Rightarrow \text{eqF } (\text{ap}_1 \text{ thmT } (\text{cgFun } v_2)) \text{codeFalse}), \quad v_2 = \text{var } 2.$$

At v_2 all three substitution-closedness witnesses are reflexivity (substT on $\text{var } 2$ leaves it fixed since $0, 1 \neq 2$), so $\text{closeW } v_2 = v_2$ definitionally.

5 Assembling the headline

The headline $\text{chaitinG1_Fun1_all}$ combines the two pieces at the fresh witness and then instantiates. The construction:

1. At $v_2 = \text{var } 2$ we have $\text{closeW } v_2 = v_2$, hence $\text{gFact } v_2 : \text{Deriv } (\text{eqF } (\text{ap}_1 \text{ G } v_2) (\text{cgFun } v_2))$ — the program image *is* the Chaitin construction at v_2 .
2. Re-index the conclusion’s subject from $\text{cgFun } v_2$ to $\text{ap}_1 \text{ G } v_2$ under thmT : from $\text{cong}_1 \text{ thmT } (\text{ruleSym } (\text{gFact } v_2))$ and ax.eqTrans one builds $\text{Deriv } (\text{eqF } (\text{thmT } (\text{cgFun } v_2)) \text{codeFalse} \Rightarrow \text{eqF } (\text{thmT } (\text{ap}_1 \text{ G } v_2)) \text{codeFalse})$, and composes it after cgFalseImpAtVar2 (via compl), yielding the implication at v_2 with subject $\text{ap}_1 \text{ G } v_2$.
3. Apply $\text{ruleInst } (2 := w)$. Because v_2 is the sole occurrence and $\text{ap}_1 \text{ G}$ is mere data,

$$\text{substT } 2 \text{ } w (\text{ap}_1 \text{ G } v_2) = \text{ap}_1 \text{ G } (\text{substT } 2 \text{ } w v_2) = \text{ap}_1 \text{ G } w,$$

and the antecedent specialises to the firing condition at w . The result holds for *every* w .

Remark 2 (Why no closedness is needed). *A genuine Fun_1 program is what removes the closedness bookkeeping. The substitution $\text{ruleInst } (2 := w)$ commutes cleanly through the data node $\text{ap}_1 \text{ G}$, so the subject becomes exactly $\text{ap}_1 \text{ G } w$ for arbitrary w . A meta-level term transformer would embed $\text{closeW } w$ in its internal slots, and substituting $v_2 \mapsto w$ into the transformer’s value at v_2 would not reproduce its value at w ; that mismatch is precisely what forced closedness hypotheses in any term-transformer formulation. With an honest combinator G , the firing point is a fresh variable and the conclusion follows for all w by one universal instantiation.*

6 The subtlety: the internal evaluator and majorisation

The conclusion ultimately rests on an *object-level* run of the universal interpreter `evalU` on the looping program g_{L^*} embedded in G . This is where the formalisation is delicate.

6.1 Representation of the universal interpreter

Inside `BRA4`, `evalU` is realised as a small CK-machine: configurations carry a control Term together with a continuation stack, and a one-step transition function `stepU` advances one rule at a time. Running the interpreter for fuel n is iteration of `stepU` from a starting configuration. A second layer wraps the μ -search loop of g_{L^*} , so that the interpreter’s outer behaviour matches Chaitin’s informal description “enumerate `thmT 0, thmT 1, ...` until a fitting proof is found”. The rule-by-rule layout of `stepU` is not what makes the proof delicate; what matters is the *shape* of the completeness statement the proof needs.

6.2 Gandy/Howard majorisation for Term-fuel completeness

A first attempt at a completeness statement would say: for every *numeral* fuel $\bar{n}$ large enough, iterating `stepU` on g_{L^*} at $\bar{n}$ reaches the final configuration `cfgRT (num  $x'$ )  $K_0$` . Such a statement is useless inside an internal `Deriv`-argument: `thmT`, `evalU`, and `stepU` all take Terms, not meta-Nats, so the fuel slot of the resulting `thmT`-fact must itself be a Term. The completeness theorem actually needed has the form

$$(x : \text{Term}) \rightarrow \text{Deriv}(\text{iter stepU} (\text{cfgEV} (\text{mcode } f) x K_0) (\text{fuelG } f x) = \text{cfgRT} (\text{num } (f x)) K_0),$$

i.e. Term-input/Term-fuel completeness, where `fuelG  $f$` is a closed BRA combinator. Proving such a statement by structural induction on the combinator tree of f stalls at the R -recursion node: the inner induction would need numerical control over how much fuel the recursive calls consume, but the actual running time is not available at the `Deriv`-level.

The resolution is the standard Gandy/Howard majorisation argument. One builds, by primitive recursion on the structure of f , a closed BRA combinator `fuelG  $f$  : Fun1` that provably *majorises* the numerical fuel any actual `stepU`-run consumes at every Term input. The induction-with-substituted-motive then carries the stable shape “the run completes at *any* fuel $\geq \text{fuelG } f x$ ”, which descends through the R -node because `fuelG` is itself defined by a recursive scheme that mirrors f . The μ -loop wrapping g_{L^*} requires the same idea one level higher: a majorising combinator `fuelMu` controls the outer search depth and is layered on top of `fuelG` by the fuel-composition lemma `iter_add_T`. The closed Term fuel that finally appears inside `ap1 G w` is

$$n_T := \text{fuelMu } k_{\max} k_{\max} + \text{fuelG } k_{\max} O, \quad k_{\max} = \text{gRec } O (s (\text{closeW } w)),$$

and the resulting positive `thmT`-fact

$$\text{thmT cPos} \stackrel{\text{Deriv}}{=} \text{coderunProg } g_L n_T = s x'$$

follows from this run via Theorem 13 (singular). Without majorisation there would simply be no Term to put in the fuel slot.

6.3 Closing the chain

With Term-fuel completeness in hand, the operational core closes the chain

$$\text{evalU}(\text{parse gLnameDef}) n_T = s(\text{outKdef}_{L^*} k_{\max})$$

through seven internal segments (the module `ChainKdef`), glued by `iter_add_T`. The positive `thmT`-fact above is its internalisation via Theorem 13. The negative leg `thmT(cmp outerWrap cSize) = cNeg coderunProg g_L n_T = s x'` is obtained from the firing condition by two `thmT_at_sb` substitutions (`passK`) and an `encoded_mp` against the size atom. The clash is then a single `encoded_mp / encoded_exfalse` pair, internal throughout under the $\Rightarrow$ -context.

6.4 No consistency hypothesis

Chaitin’s original argument concludes “ T is inconsistent” (in the metatheory). The BRA4 formalisation strengthens this: the conclusion is an *internal* implication `Deriv (X(w)  $\Rightarrow$  eqF (thmT (ap1 G w)) codeFalse)`, i.e. a single BRA derivation that, on the firing condition, the constructed code `ap1 G w` is a T -proof of $0 = 1$. No `Con(T)` premise is consumed, no `isNat` integrality predicate is consumed, and the firing condition is stated in `num-row` form.

This is the precise interface offered to the later surprise-Gödel-II descent: that descent will *not* need to reprove any evaluator completeness, and will use the implication purely as a black-box arrow from a `Deriv`-witness of the firing condition to a `Deriv`-witness of `codeFalse`. The Term-fuel completeness work paid for here — and in particular the Gandy/Howard machinery of §6.2 — is invisible to the descent. The contrast with consistency is sharp: the implication itself consumes no `Con(T)`, but the surprise-Gödel-II argument will consume internal consistency *after*, and only after, the implication has been discharged.

7 Files

- `BRA4/AbsFun1.agda` — the one-variable bracket-abstraction DSL: `Exp`, `denote`, `compile`, and the combinatory-completeness evaluation `compile_eq` (a proved `Deriv`).
- `BRA4/CgFunFun1.agda` — the genuine `Fun1` diagonal `G = compile bodyExp`, its defining fact `gFact`, and the headline `chaitinG1.Fun1_all` (plus the R -carrying and the closed-witness corollaries).
- `BRA4/CgFalseImp.agda` — the internal implication `cgFalseImp` (deduction-theorem internalisation), its un-parametrised form `cgFalseImpDed`, and the fresh-witness instance `cgFalseImpAtVar2`.
- `BRA4/CloseW.agda` — `closeW` and the structural lemmas it supports.

All files typecheck under `--safe --without-K --exact-split` with no holes and no postulates.

References

- [1] G. J. Chaitin, “Computational complexity and Gödel’s incompleteness theorem,” *ACM SIGACT News*, no. 9, pp. 11–12, 1971.
- [2] G. J. Chaitin, “Information-theoretic limitations of formal systems,” *Journal of the ACM*, vol. 21, no. 3, pp. 403–424, 1974.